

TP5 Partage du bus dans les architectures multi-processeurs

B. Architecture matérielle

Question de soi-même : que signifie enum ?

enum permet de créer un type qui peut contenir des constantes actualisables.

Il faut modifier le niveau d'associativité et le nombre de ligne d'un cache dans le fichier tp5_top.cpp

Question B1: En consultant l'en-tête du fichier pibus_frame_buffer.h, précisez la signification des arguments du constructeur du composant PibusFrameBuffer.

- **name** : Le nom de l'instance du module.
- **tgtdid** : L'identifiant (index) de la cible sur le bus.
- **segtab** : La table des segments pour l'adressage.
- **latency** : Le temps d'accès intrinsèque du composant.
- **width / height** : La résolution de l'écran (ici 256x256).
- **subsampling** : Le format de codage des pixels (par défaut 420).

Question B2 : quelle est la longueur du segment associé à ce composant ?

- **Taille d'un tampon** : $256 \text{ lignes} \times 256 \text{ pixels/ligne} \times 1 \text{ octet/pixel} = 65536 \text{ octets}$ (soit **64 Ko**).
- **Nombre de tampons** : Le composant contient deux tampons de 64 Ko chacun.
- **Total** : $64 \text{ Ko} + 64 \text{ Ko} = 128 \text{ Ko}$.

C. Compilation de l'application logicielle

Question C1 : Pourquoi faut-il utiliser un appel système pour accéder (en lecture comme en écriture) au contrôleur de frame-buffer ? Que se passe-t-il si un programme utilisateur essaie de lire ou d'écrire directement dans le Frame Buffer, sans passer par un appel système ?

- Pour assurer la protection et l'abstraction. Le matériel (I/O) est mappé dans un espace mémoire protégé (noyau).
- Si accès direct ? Le processeur déclenche une exception de type Bus Error ou Segmentation Fault, car le programme utilisateur n'a pas les privilèges pour accéder aux adresses physiques des périphériques.

Question C2 : Pourquoi préfère-t-on construire une ligne complète dans un tableau intermédiaire de 256 pixels plutôt que d'écrire directement l'image - pixel par pixel - dans un dans le frame buffer ?

L'écriture pixel par pixel générerait $256 \times 256 = 65536$ transactions WRITE sur le bus. Chaque transaction possède un surcoût (arbitrage, adresse). En construisant une ligne complète en RAM puis en utilisant `fb_sync_write()`, on optimise l'utilisation du bus (possibilité de transferts en rafale ou bloc)

Question C3 : Consultez le fichier `stdio.c`, et expliquez la signification des trois arguments de l'appel système `fb_sync_write()`. Complétez le fichier `main.c` en conséquence.

L'appel système `fb_sync_write()` permet de transférer des données depuis la mémoire RAM vers Frame Buffer. + efficace de la bande passante du bus

Il permet de laisser le MIPS continuer sur d'autres tâches.

1. `offset` : Numéro de la ligne à écrire (0 à 255)
2. `buffer` : Pointeur vers le tableau de données en RAM
3. `length` : Nombre de pixels à transférer

D. Caractérisation de l'application logicielle

Question D1 : Quel est le nombre de cycles nécessaires pour afficher l'image avec un seul processeur. Combien d'instructions ont été exécutées pour afficher l'image? Quel est le nombre moyen de cycles par instruction (CPI)?

Nombre de cycles : cycles

Nombre instructions : 4003722

CPI : 1.37372

*** proc[0] at cycle 5500000

- INSTRUCTIONS = 4003722
- CPI = 1.37372
- CACHED READ RATE = 0.241966
- UNCACHED READ RATE = 0.00132751
- WRITE RATE = 0.126301
- IMISS RATE = 0.00924989
- DMISS RATE = 0.0106868
- IMISS COST = 15.917
- DMISS COST = 14.7441
- UNC COST = 6
- WRITE COST = 0

bcu : Statistics

master 0 : n_req = 558376 , n_wait_cycles = 558376 , access time = 1

master 1 : n_req = 0 , n_wait_cycles = 0 , access time = -nan

master 2 : n_req = 0 , n_wait_cycles = 0 , access time = -nan

Question D2 : Quel est le pourcentage d'écritures sur l'ensemble des instructions exécutées ? Quel est le pourcentage de lectures de données ? Quels sont les taux de miss sur le cache instruction et sur le cache de données ? Quel est le coût d'un miss sur le cache instructions ? Quel est le coût d'un miss sur le cache de données ? Quel est le coût d'une écriture pour le processeur ? On rappelle que les coûts se mesurent en nombre moyen de cycles de gel du processeur. Comment expliquez-vous que ces coûts ont des valeurs non entières?

i) WRITE RATE = 0.126301

ii)

- CACHED READ RATE = 0.241966
- UNCACHED READ RATE = 0.00132751

iii)

- WRITE RATE = 0.126301
- IMISS RATE = 0.00924989

iv) IMISS COST = 15.917

v) DMISS COST = 14.7441

vi) WRITE COST = 0

Le coût du miss dépend de l'attente de l'autorisation du bus. De ce fait, le temps d'attente d'une autorisation du bus dépend de la situation actuel

Question D3 : Évaluez le nombre de transactions de chaque type pour cette application. Que remarquez-vous?

Répartition des transactions sur le bus:

Quantité de transaction write : $\text{instructions} * \text{write_rate} = 4\,003\,722 * 0.126301 = 505\,674$
instructions write.

Quantité de transactions read = $\text{instructions} * (\text{CACHED READ RATE} + \text{UNCACHED READ RATE}) = 4\,003\,722 * (0.241966 + 0.00132751) = 974\,079$ instructions read.

E. Exécution sur architecture multi-processeurs

Le numéro est une valeur câblée au moment de la fabrication dans le registre protégé

-> `procid()`

Question E2 : Pourquoi les piles d'exécution de N programmes s'exécutant sur les N processeurs doivent-elles être strictement disjointes?

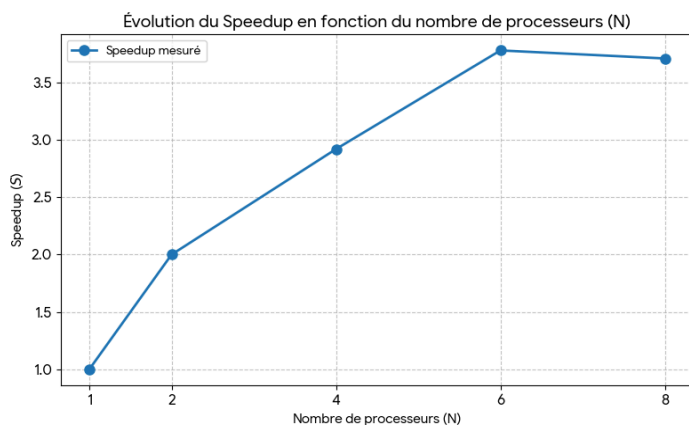
Chaque processeur exécute ses propres appels de fonctions et stocke ses variables locales. Si les piles se chevauchent, un processeur écrasera les données de l'autre (corruption de pile)

Question E3 : Donnez deux raisons pour lesquelles le code binaire doit être recompilé chaque fois qu'on change le nombre de processeurs.

- On modifie le nombre de processeurs sur le fichier `config.h` qui impacte sur la boucle dépendant de `NPROCS` sur `main.c`. De ce fait, on doit le recompiler pour le mettre à jour.
- **Raison 2 : L'initialisation des piles d'exécution au niveau système (dans `reset.s`)** Comme vu à la question E2, chaque processeur doit posséder sa propre pile d'exécution (stack) dans la mémoire pour stocker ses variables locales sans écraser celles des autres processeurs. Le code de démarrage (`reset.s`) utilise le numéro du processeur et le nombre total de processeurs prévu pour décaler le pointeur de pile (`$sp`) de chaque processeur à la bonne adresse mémoire (par tranches de 64 Ko). Changer le nombre de processeurs modifie l'organisation spatiale de ces piles.

Question E4 Remplissez le tableau ci-dessous, et représentez graphiquement `speedup(N)` en fonction de N.

	1 proc	2 proc	4 proc	6 proc	8 proc
cycles	5 280 688	2 636 327	1 805 449	1 396 641	1 421 608
speedup	1	2	2.92	3.78	3.71



F. Evaluation des temps d'accès au bus

printStats() -> nombre moyen de cycle de gel par proc

printStats() au BCU -> temps d'accès moyen au bus = cycle attente

→ on divise le nombre total de cycles d'attente du maître par le nombre total de requêtes effectuées par le maître.

Question F1 : Pourquoi faut-il absolument effectuer la mesure au moment précis où l'application se termine?

Si vous attendez trop longtemps (le risque principal) : Dans une architecture multi-processeurs, lorsqu'un processeur a terminé sa part du travail, il exécute la fonction `exit()` et rentre dans une boucle d'attente infinie (souvent appelée *idle loop*). Pendant qu'il tourne dans le vide en attendant que la simulation globale s'arrête, il continue de lire des instructions en mémoire. Cela génère un trafic constant et "inutile" sur le bus (des requêtes IMISS). Ces requêtes fantômes vont artificiellement gonfler les compteurs et fausser complètement vos moyennes de temps d'accès, vos coûts et votre CPI.

Si vous arrêtez trop tôt : Vous n'aurez pas mesuré l'exécution complète de l'algorithme, et les statistiques pourraient être biaisées par la phase de démarrage (qui comporte généralement beaucoup plus de *cache misses* que la phase de calcul de l'image).

Question F2 Remplissez le tableau ci-dessous, en exécutant successivement le programme sur différentes architectures:

NPROCS	1	2	4	6	8
IMISS COST	15.89	16.58	23.04	41.43	70.29
DMISS COST	14.74	15.54	21.86	36.94	61.48
WRITE COST	0.0000	0.00178	0.763	4.21	6.89
ACCESS_TIME	1	1.5	4.75-4.9	11.10	16.3
CPI	1.394	1..392	1.44	2.27	1.51

anomalie sur le CPI pour 8 processeurs, il devrait être supérieur à 6 processeurs.

Question F3 : Interprétez ces résultats. La dégradation de la valeur du CPI quand on augmente le nombre de processeurs est-elle principalement due à l'augmentation du coût des MISS, ou à l'augmentation du coût des écritures?

- L'augmentation du nombre de processeurs fait augmenter le nombre de requêtes d'accès au bus passant de 1 cycle à 16 cycles. Le bus est saturé.
- **L'impact majeur sur les MISS (IMISS / DMISS) :** Lorsqu'un processeur subit un défaut de cache (miss), il est totalement bloqué. Il **doit** attendre que la donnée ou l'instruction revienne de la mémoire principale (RAM) pour continuer à travailler. Puisque le temps d'attente sur le bus explose, le coût d'un MISS explose avec lui (passant de ~15 cycles à plus de 60-70 cycles). C'est ce qui pénalise le plus le CPI.
- **Le rôle tampon pour les écritures (WRITE COST) :** le coût d'une écriture est de **0 cycle** avec un seul processeur. Le processeur "poste" son écriture dans le tampon et continue immédiatement son travail sans attendre que la donnée traverse le bus. Le coût des écritures ne commence à augmenter (vers 4 à 7 cycles) que lorsque le bus est tellement saturé que le tampon n'arrive plus à se vider et finit par déborder, obligeant le processeur à geler. Même à 8 processeurs, la pénalité d'une écriture (~7) reste négligeable comparée à celle d'un MISS (~70).

G. Modélisation du comportement du bus

Question G1 : En exploitant votre compréhension du protocole PIBUS, calculez le nombre de cycles d'occupation du bus pour chacun des 4 types de transaction. Attention : il ne faut pas compter le temps d'attente pour accéder au bus, car la phase REQ->GNT n'utilise pas le bus et a lieu pendant la fin de la transaction précédente. Par contre, il faut compter un cycle supplémentaire correspondant au "cycle mort" entre deux transactions.

- 1 cycle de commande : Le maître envoie l'adresse cible et le type de commande sur le bus.
- N cycles de données : Le transfert des mots d'information. La RAM et le composant FBF n'ont pas de latence, et le transfert est de 1 mot par cycle.
- 1 cycle mort entre deux transactions

Formule : 1 (Commande) + N (nombre de mots) + 1 (Cycle mort)

Une ligne est composée de 8 mots.

- **IMISS** : $1 + 8 + 1 = 10$ cycles
- **DMISS** : $1 + 8 + 1 = 10$ cycles
- **UNC** : $1 + 1 + 1 = 3$ cycles
- **WRITE** : $1 + 1 + 1 = 3$ cycles

Les temps d'occupation théoriques du bus sont de 10 cycles pour les rafales et de 3 cycles pour les accès simples.

Question G2 : En exploitant les résultats de la partie D (pourcentage de lectures cachées, pourcentage d'écritures, taux de MISS sur les caches, valeur du CPI), calculez la fréquence de chacun des 4 types de transaction et complétez le tableau ci-dessous. Puisque notre unité de temps est le cycle, ces fréquences seront exprimées en nombre d'événements par cycle.

	Temps_Occupation	Fréquence
IMISS	15.917	0.00924989
DMISS	14.7441	0.0106868
UNC	6	0.00132751
WRITE	0	0.126301

Question G3 : En utilisant les résultats des deux questions précédentes, calculez le pourcentage de la bande passante du bus utilisé par un seul processeur. En déduire le nombre de processeurs au-delà duquel le bus commence à saturer.